

Sequential Search using Sentinel

1 What is Sequential Search?

Sequential search is a simple searching technique in which the elements of an array are checked **one by one** from the beginning until the required element, called the **key**, is found.

It is also called **Linear Search**.

For example, consider:

$$A = [10, 25, 30, 45, 60]$$

Suppose the key is:

$$key = 45$$

The search proceeds as follows:

$$10 \neq 45$$

$$25 \neq 45$$

$$30 \neq 45$$

$$45 = 45$$

Therefore, the key is found at index 3.

2 Basic Linear Search

The basic linear search algorithm is:

```
1 for(int i = 0; i < n; i++)
2 {
3     if(A[i] == key)
4     {
5         return i;
6     }
7 }
8
9 return -1;
```

There are two important checks in this algorithm:

1. Check whether the current index is within the array.
2. Check whether the current element is equal to the key.

The two conditions are:

$$i < n$$

and

$$A[i] == key$$

Therefore, the algorithm has to repeatedly check both the **array boundary** and the **key**.

3 Why Do We Need a Sentinel?

Consider:

$$A = [10, 25, 30, 45, 60]$$

Suppose the key is:

$$key = 50$$

The key is not present in the array.

In ordinary linear search, we compare:

$$10 \neq 50$$

$$25 \neq 50$$

$$30 \neq 50$$

$$45 \neq 50$$

$$60 \neq 50$$

After checking the last element, we must stop the search.

Therefore, the algorithm needs the additional condition:

$$i < n$$

This boundary checking is repeated during the search.

The idea of a **sentinel** is to add the key itself at the end of the array.

4 Adding the Key as a Sentinel

Consider the original array:

$$A = [10, 25, 30, 45, 60]$$

and:

$$key = 50$$

Instead of searching only the original array, we temporarily add the key as a new element at the end.

Therefore:

$$A = [10, 25, 30, 45, 60, \boxed{50}]$$

The newly added 50 is called the **sentinel**.

The sentinel is not an original element of the array.

It is added only to guarantee that the search will stop.

Therefore:

$$\text{Sentinel} = \text{Key inserted at the end as a stopping value}$$

5 How Does Sentinel Search Work?

After adding the sentinel:

$$A = [10, 25, 30, 45, 60, 50]$$

The search starts from the first element.

$$10 \neq 50$$

$$25 \neq 50$$

$$30 \neq 50$$

$$45 \neq 50$$

$$60 \neq 50$$

Finally:

$$50 = 50$$

The search stops.

Notice that we did not need to ask:

$$i < n$$

inside the search loop.

Why?

Because the key has been deliberately placed at the end.

Therefore, the search is guaranteed to stop.

6 Sentinel Search Algorithm

The basic idea is:

```
1 Add key at the end of the array
2
3 i = 0
4
5 while(A[i] != key)
6 {
7     i++;
8 }
9
10 if(i < n)
11     key is found
12 else
13     key is not found
```

Here, the original array contains n elements.

The sentinel is added at position:

$$n$$

Therefore:

$$A[n] = \text{key}$$

If the search stops at:

$$i < n$$

then the key was present in the original array.

If the search stops at:

$$i = n$$

then the key found was only the sentinel.

Hence:

$i = n \Rightarrow \text{Key was not originally present}$

7 Number of Comparisons in Linear Search

There are two types of comparisons in the ordinary linear search.

7.1 1. Key Comparison

At every element we check:

$$A[i] == key$$

For $n = 5$:

5 key comparisons

7.2 2. Boundary Comparison

The loop also checks:

$$i < n$$

For a loop that runs through all 5 elements, the condition is checked one extra time when the loop terminates.

Therefore:

6 boundary comparisons

Hence, if we count both types of comparisons:

Total comparisons = 5 + 6

$$\boxed{11}$$

Thus, ordinary linear search performs:

$$\boxed{11 \text{ condition checks}}$$

for this particular unsuccessful search.

8 Now Add the Sentinel

Original array:

$$A = [10, 25, 30, 45, 60]$$

Key:

$$key = 50$$

Add the key as a new element:

$$A = [10, 25, 30, 45, 60, \boxed{50}]$$

The last 50 is the sentinel.

Now perform:

Index	$A[i]$	$A[i] == key$
0	10	No
1	25	No
2	30	No
3	45	No
4	60	No
5	50	Yes

The search stops automatically when it reaches the sentinel.

9 Number of Comparisons in Sentinel Search

There are now:

6

key comparisons:

10, 25, 30, 45, 60, 50

Therefore:

$$6 \text{ key comparisons} + 1(i < n) = 7 \text{ comparisons}$$

But notice something important.

There is **no separate boundary comparison** inside the search loop.

The loop can simply be:

```

1 while(A[i] != key)
2 {
3     i++;
4 }
```

Therefore, the repeated boundary checking:

$$i < n$$

has been eliminated.

10 Comparison of Both Methods

For:

$A = [10, 25, 30, 45, 60]$

and:

$key = 50$

we get:

Comparison	Linear Search	Sentinel Search
Key comparisons	5	7
Boundary comparisons	6	0
Total condition comparisons	11	6

Therefore:

$$\text{Linear Search} = 5 + 6 = 11$$

while:

$$\text{Sentinel Search} = 7$$

The sentinel search performs one additional key comparison because the sentinel itself must be compared with the key.

However, it eliminates the repeated boundary checking.

11 Why is the Sentinel Significant?

The main significance of the sentinel is:

1. It guarantees that the search will terminate.
2. It eliminates the need to check the array boundary during every iteration.
3. It simplifies the search loop.
4. It replaces two checks with one check.

Ordinary linear search:

```
1 while(i < n && A[i] != key)
2 {
3     i++;
4 }
```

Sentinel search:

```
1 while(A[i] != key)
2 {
3     i++;
4 }
```

Therefore:

Sentinel removes the boundary check from the search loop

12 Why Does Adding the Key Guarantee Termination?

Suppose the key is not present in the original array.

Without a sentinel:

$10 \rightarrow 25 \rightarrow 30 \rightarrow 45 \rightarrow 60$

After 60, there is no key.

Therefore, we need:

$$i < n$$

to stop the search.

With a sentinel:

$10 \rightarrow 25 \rightarrow 30 \rightarrow 45 \rightarrow 60 \rightarrow 50$

The key is guaranteed to occur at the end.

Therefore:

The search must stop at or before the sentinel.

This is the main idea behind sentinel search.

13 Important: Sentinel Does Not Change Big-O

Although sentinel search reduces the number of checks inside the loop, it does not change the asymptotic time complexity.

For ordinary linear search:

$O(n)$

For sentinel search:

$O(n)$

Therefore:

Linear Search and Sentinel Search are both $O(n)$

The difference is in the **number and type of comparisons**, not the Big-O class.

14 Simple Comparison

Linear Search	Sentinel Search
Search the original array.	Add the key as a new last element.
Checks the key.	Checks the key.
Checks the boundary repeatedly.	Does not need a boundary check in the search loop.
Stops because the array boundary is reached.	Stops because the sentinel guarantees the key will be found.
Worst case: $O(n)$.	Worst case: $O(n)$.

15 Key Concept

The entire idea of sentinel search can be remembered using the following statement:

“Add the key at the end so that the search is guaranteed to stop.”

The sentinel does not mean that the key was originally present.

It is an **artificial copy of the key** added only to provide a guaranteed stopping point.

Sentinel = Artificial stopping value
